
boolean



DATABASE

Data persistence:
come salvare dati in maniera persistente

Database: a cosa serve?

È un sistema tecnologico che permette di
gestire una collezione organizzata di dati coerenti.

Le caratteristiche fondamentali sono:

- Permettere le operazioni di **creazione, cancellazione**, aggiornamento e **modifica** dei dati
- Offrire strumenti per la **ricerca avanzata** di dati all'interno della collezione
- Garantire il **salvataggio permanente**

Database: un po’ di storia

La necessità di gestire dati in maniera organizzata nasce con i primi mainframe.
Negli anni ‘70, Edgar Codd dell’IBM lavora su una formalizzazione (anche logico-matematica) di come poteva essere organizzato un database introducendo i **database relazionali**.

Da allora sono nati diversi software che utilizzano le regole **relazionali** per fornire le funzionalità di un **Database Management System** (MySQL, MS SQL Server, IBM DB2, ...)







Database: terminologia

1. Un **database** è una collezione organizzata di dati coerenti.

2. **(R)DBMS** - **(Relational) Database Management System** è il mezzo tecnologico che permette di gestire la collezione di dati (il **database**)

3. Nella lingua parlata, il termine **database** viene utilizzato anche per riferirsi al **DBMS**

4. **NoSQL** - Sono una categoria di DBMS che non implementano le regole relazionali (es. MongoDB)

Transazioni

Le operazioni sui dati vengono eseguite in **transazioni**: una sequenza di operazioni che, se eseguita in modo corretto, produce una variazione.

In caso di successo, il risultato delle operazioni deve essere permanente o persistente.
In caso di insuccesso si deve tornare allo stato precedente all'inizio della transazione.



ACID properties

Proprietà acide...

Le transazioni in un RDBMS hanno le seguenti proprietà:

1 **Atomicità:** la transazione è indivisibile nella sua esecuzione e la sua esecuzione deve essere o totale o nulla, non sono ammesse esecuzioni parziali

2 **Coerenza:** non devono verificarsi contraddizioni (incoerenza) tra i dati archiviati nel DB

3 **Isolamento:** ogni transazione deve essere eseguita in modo isolato e indipendente dalle altre transazioni, l'eventuale fallimento di una transazione non deve interferire con le altre transazioni in esecuzione

4 **Durabilità:** (o persistenza): una volta che una transazione abbia richiesto un commit work, i cambiamenti apportati non dovranno essere più persi.

Database Relazionali

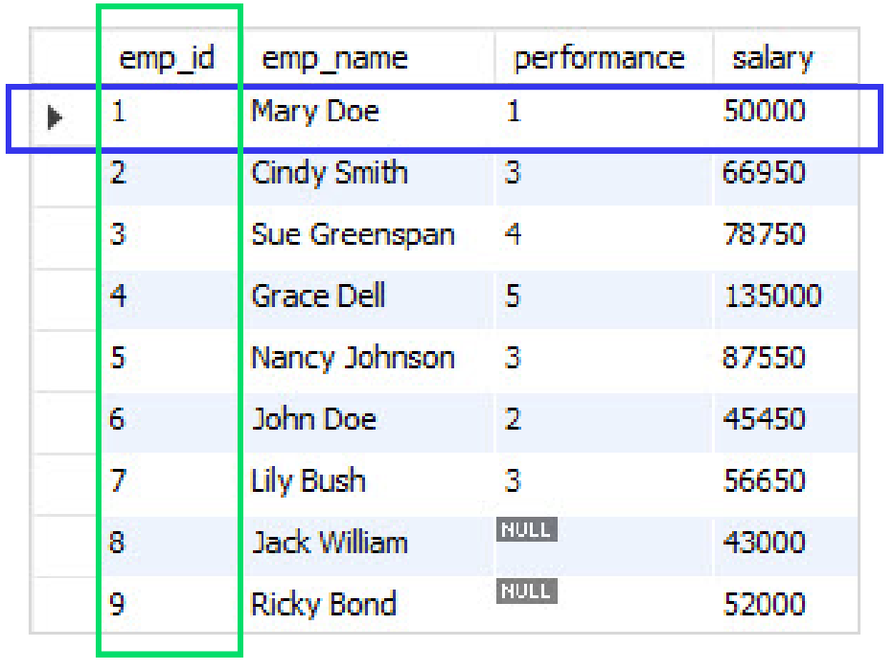
I DB relazionali organizzano i dati in tabelle.

Una tabella di un database è formata da
righe (o tuple) e **colonne**

Le colonne rappresentano la struttura, ossia le caratteristiche del dato da memorizzare.
Ogni riga rappresenta un dato inserito.

Per creare una tabella dobbiamo decidere la sua struttura, che comprende:

- Nome Tabella
- Colonne
- Indici



emp_id	emp_name	performance	salary
1	Mary Doe	1	50000
2	Cindy Smith	3	66950
3	Sue Greenspan	4	78750
4	Grace Dell	5	135000
5	Nancy Johnson	3	87550
6	John Doe	2	45450
7	Lily Bush	3	56650
8	Jack William	NULL	43000
9	Ricky Bond	NULL	52000



LIVE CODING

Immaginiamo la struttura di una tabella

Colonne in un Database

La scelta di quale e quante colonne non è così banale come può sembrare: tralasciando i discorsi di “normalizzazione”, **scegliere correttamente** dal principio i dati che le colonne devono contenere è fondamentale per evitare costosi cambiamenti nel futuro.

È importante scegliere:

- Nome della colonna
  - Chiaro e immediatamente comprensibile
  - Future-proof
- Rappresentazione del dato da inserire
  - Es. una data di nascita (17-02-1988, 17th Feb 88, ...)
- Tipo di dato
  - Con cui il database gestirà l’informazione
- Attributi
  - Particolari caratteristiche per ogni colonna


https://www.studytonight.com/dbms/database-normalization.php

Che tipi di dato possiamo salvare in un DB?

Qualsiasi cosa!

I database relazionali supportano nativamente
diversi tipi di dato che possiamo raggruppare in:

1. **Numeri**
2. **Stringhe**
3. **Date**
4. **Altri meno utilizzati**
    1. Dati geometrici/geografici
    2. Dati binary (BLOB)

Numeri nel Database

Numeri interi

- **TINYINT**
  Utilizza solo un byte per salvare numeri che vanno da -128 a 127.
  Viene utilizzato ad esempio per salvare i valori boolean, proprio perché è il tipo di dato più piccolo possibile
- **SMALL / MEDIUMINT**
  Occupano rispettivamente 2 e 3 bytes
- **INT**
  Utilizza 4 bytes per rappresentare numeri compresi tra -2.147.483.648 e 2.147.483.647
- **BIGINT**
  Corrisponde ad un INT con il doppio di bytes disponibili

Numeri nel Database

Numeri con la virgola

- **FLOAT(I, D)**
  un numero con la virgola da 4 bytes
- **DOUBLE(I, D)**
  un numero con la virgola da 8 bytes
- **DECIMAL(I, D)**
  un double che gestisce gli arrotondamenti considerando i numeri dal punto di vista finanziario

I e **D** identificano:



- il numero di cifre totali (I)
- quante di queste ne vogliamo dedicare alla parte decimale (D).

Ad esempio, utilizzando un float(6,4), il numero 56,01528012 diverrà 56,0153 proprio perché potrà avere al massimo 6 numeri, di cui 4 dopo la virgola.

Numeri nel Database

Choose Wisely

Nel 2014, **YouTube** ha dovuto cambiare il tipo di dato utilizzato per contare le views dei suoi video quando **GangNam Style** si stava avvicinando alla soglia delle **2,147,483,647** views, limite massimo per un numero a 4byte, portandolo a 8byte.

Ora il limite è **9,223,372,036,854,775,807**



Stringhe nel Database

Oltre ai numeri, ovviamente, possiamo salvare anche dati di tipo stringa

Anche in questo caso ciò che cambia è la capacità di ogni singolo tipo:

- **CHAR(numero)**
  permette di indicare la lunghezza esatta della stringa da rappresentare, con il limite di 255 caratteri, risparmiando 1 byte
- **VARCHAR(numero)**
  permette di indicare la lunghezza massima della stringa da rappresentare, con il limite di 255 caratteri. Consuma 1 byte in più per la flessibiltà
- **TEXT**
  fino a 65535 caratteri - è usato ad esempio per le descrizioni prodotto
- **MEDIUMTEXT**
  16mb
- **LONGTEXT**
  4.2gb - usato per salvare porzioni di HTML o lunghi testi

Stringhe nel Database

Differenza tra CHAR e VARCHAR

Questa tabella ci mostra la differenza di consumo tra **VAR** e **VARCHAR** se inseriamo una stringa di 4 caratteri. Come si nota, se sappiamo che i valori della nostra colonna saranno esattamente 4, ci conviene impostare un **CHAR(4)** e risparmiare 1 byte! Altrimenti, sfruttiamo la flessibilità di **VARCHAR**

Value	CHAR(4)	Storage Required	VARCHAR(4)	Storage Required
''	' '	4 bytes	''	1 byte
'ab'	'ab '	4 bytes	'ab'	3 bytes
'abcd'	'abcd'	4 bytes	'abcd'	5 bytes
'abcdefgh'	'abcd'	4 bytes	'abcd'	5 bytes

Date nel Database

I tipi di dato per le date

- **DATETIME**
  permette di memorizzare data e ora (YYYY-MM-GG HH:II:SS)
- **DATE**
  contiene solo la data (YYYY-MM-GG)
- **TIME**
  contiene solo l'orario (HH:II:SS)
- **YEAR**
  contiene solo l'anno (YYYY)
- **TIMESTAMP**
  può avere diversi formati



Perché tutta questa fatica?

Non potrei fare tutto **BIGINT**, **DATETIME**, **LONGTEXT** “per stare sul sicuro”?



LIVE CODING

Aggiungiamo alla tabella precedente i tipi di dati

Primary Key

O chiave primaria o identificatore

Ogni tabella deve avere una **colonna adibita ad identificatore**, cioè un valore diverso per ogni riga inserita, che permetta di individuarla univocamente.

Il caso più semplice e comune è avere una **colonna id** che contenga un numero intero che cresce automaticamente all'inserimento di ogni nuova riga

ID	NAME	CLASS	MARK	SEX
1	John Deo	Four	75	female
2	Max Ruin	Three	85	male
3	Arnold	Three	55	male
4	Krish Star	Four	60	female
5	John Mike	Four	60	female
6	Alex John	Four	55	male
7	My John Rob	Fifth	78	male
8	Asruid	Five	85	male
9	Tes Qry	Six	78	male
10	Big John	Four	55	female

Primary Key

O chiave primaria o identificatore

In rari casi, un identificatore può essere anche una **coppia di due colonne**

firstname	lastname	email
Alejandro	Gervasio	alejandro@domain.com
John	Doe	john@domain.com
Susan	Norton	susan@domain.com
Marian	Wilson	marian@domain.com
Mary	Smith	mary@domain.com
Amanda	Bears	amanda@domain.com
Jodie	Foster	jodie@domain.com
Laura	Linney	laura@domain.com
Alice	Dern	alice@domain.com
Jennifer	Aniston	jennifer@domain.com

Attributi

Ad ogni colonna, possiamo assegnare degli attributi

- **NULL / NOTNULL**
  indica che la colonna può o non può contenere il valore NULL.
  Se una colonna è NOT NULL e durante il salvataggio non viene passato alcun valore per quella colonna, verrà restituito un errore e l'intera riga non verrà inserita.
  Esempio:
  In un database di un'anagrafe, la colonna data_di_nascita sarà NOT NULL per indicare che non è possibile salvare nel database di una persona senza indicare anche la data di nascita
- **DEFAULT**
  permette di settare un valore di default alla colonna qualora non fosse presente alcun valore al momento del salvataggio.
  Esempio:
  Se vogliamo contare il numero di volte in cui un prodotto è stato acquistato, può avere senso avere DEFAULT(0)

Altri Attributi...

- **AUTO_INCREMENT**
  Il valore della colonna è incrementa ad ogni nuovo record, per questo motivo è utilizzabile solo con le colonne di tipo numero.
  Viene utilizzato ad esempio per gli ID: ogni volta che si aggiunge una nuova riga, in automatico l'ID sarà +1 rispetto alla riga precedente.
- **UNIQUE**
  Indica che i valori di quella colonna devono essere unici, ossia non ci può essere alcun valore ripetuto all'interno della colonna.



Le colonne designate a PRIMARY_KEY hanno di default gli attributi NOT NULL e UNIQUE.
Molto spesso sono anche AUTO_INCREMENT



LIVE CODING

Aggiungiamo alla tabella precedente attributi e chiave primaria